Reliable Data Transfer

Lecture 16

Computer Networks and Distributed Programming

Where we are

When we cannot rely on TCP use datagrams (UDP), reliability is to be guaranteed by hand (responsibility of the application).

- Acknowledgement, timeout, retransmission, sequence numbers: four mechanisms used for this issue.
- Question: a 50 ms timer on a path with an 80 ms round trip. What happens?
- Today the four mechamisms above come back as a general mechanism, built one channel model at a time.
- Then we lift the ceiling: one message in flight is slow, and there are exactly two ways out of it.

An announcement on an unreliable channel

You are waiting for train 6 on platform 5. The train is moved, and station control sends: *train 6 will arrive on platform 9*.

- A word is lost: *train . . . will arrive on platform 9* — readable, and useless.
- A word is altered: *train 7 will arrive on platform 9* — perfectly readable, and wrong.
- Two words are swapped: *train 9 will arrive on platform 6* — every word arrived; the meaning did not.
- A word is duplicated: *train 66 will arrive on platform 9*.

Sometimes you can tell the message is broken. Sometimes you take the wrong train — and noticing is not the same as recovering.

What a reliable channel guarantees

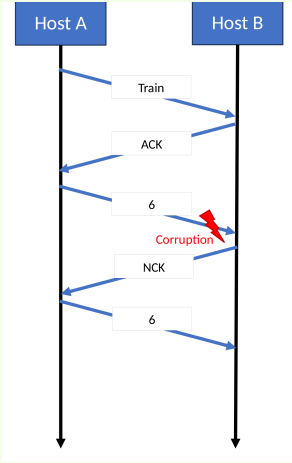
Reliable data transfer

1. No transmitted bit is corrupted — flipped from 0 to 1, or the other way.
2. No transmitted bit is lost, and none is delivered twice.
3. Everything arrives in the order in which it was sent.

- The layer underneath offers none of the three: TCP is reliable and runs on top of IP, which is not.
- So the whole job is done at the two ends, out of an unreliable point-to-point channel.
- And it is not one layer's problem: the same three guarantees are rebuilt at the link layer, and in applications.

A channel that only corrupts bits

- First model: bits may be flipped, but nothing is lost and nothing is reordered.
- Corruption comes from the physical components, while a packet is transmitted, propagated or buffered.
- Stop-and-wait: one message on the wire; the next one leaves only after the answer to this one.
- ACK: it arrived intact. NAK: something is wrong, send it again.
- Protocols that recover by retransmitting are called ARQ, Automatic Repeat reQuest.



These figures write `NCK`: the standard name is `NAK`.

What an ARQ protocol needs

- Error detection: extra bits that let the receiver notice the message changed — the checksum you have already met.
- Receiver feedback: the sender is thousands of kilometres away and can see nothing: the only way to know is to be told.
- Retransmission: whatever arrived in error is sent again. There is no other repair.

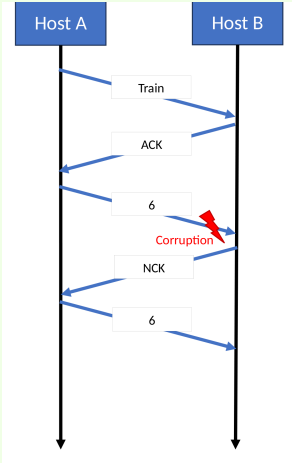
In principle the feedback is one bit: a value for *yes* and one for *no*. None of the three is optional — and, as it turns out, the three together are still not enough.

Two questions this scheme cannot answer

What if the acknowledgement itself is corrupted?

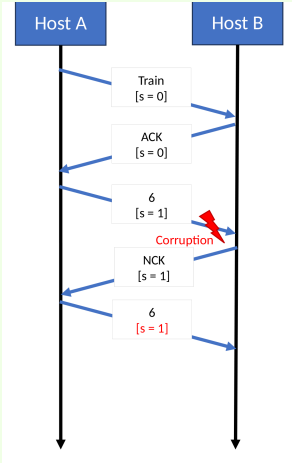
Asking again introduces a question that can also be corrupted. Resending introduces a second copy the receiver cannot tell from a new message.

- A checksum on the acknowledgement detects the damage; it does not recover what the receiver actually said.
- So the receiver has a question of its own — new message, or the previous one again? — and nothing in the message answers it.



One bit is enough

- Stop-and-wait keeps exactly one message on the wire, so there is no long sequence to describe.
- The receiver only has to tell this message from the previous one — two cases, one bit.
- Add a field $s = 0/1$ to the header: it alternates, and a repeated value means a repeated message.
- The acknowledgement carries the same bit, so an answer says which message it answers.

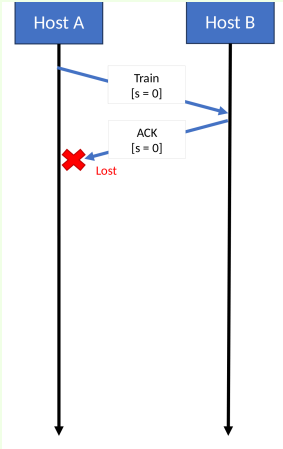


Numbered acknowledgements make NAKs unnecessary

- A NAK says *resend*. An acknowledgement that carries a number says *I have up to here*, which says more.
- If the sender receives the acknowledgement for the previous message twice, nothing has moved: that is a NAK by another name.
- So the receiver has one answer for both cases — damaged message or duplicate — and it is always an ACK, with a number.
- One packet type less, and a receiver that never has to explain what went wrong, only what it has.
- Those duplicate acknowledgements come back later: they are how TCP notices a loss without waiting for a timer.

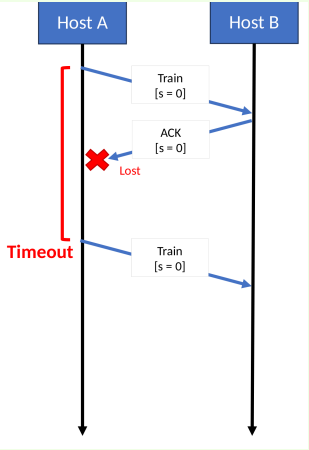
If nothing comes back, nothing moves

- Second model: packets can also be lost. You know where they go — a queue fills up and the router drops.
- Stop-and-wait now stops for good: the sender is waiting for an answer that will never be produced.
- It makes no difference which one was lost, the message or the acknowledgement.
- In both cases the sender observes the same thing — nothing — and nothing else will ever wake it up.



Timeout: the sender takes the burden

- Give the sender a clock: if no answer arrives within a chosen time, send the message again.
- One mechanism covers both losses, precisely because the sender cannot distinguish them.
- The two cases differ only at the receiver: when the acknowledgement was lost, a duplicate arrives.
- The duplicate is recognised by its sequence number, discarded, and acknowledged again.
- A DNS query over UDP does exactly this: no answer in time, ask once more.



Choosing the timeout

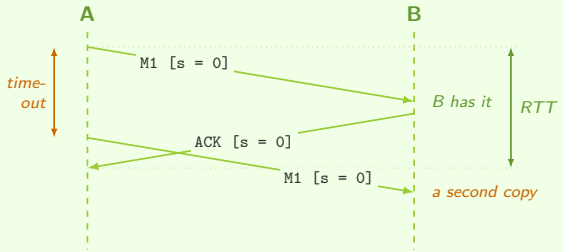
- Too long: every loss is paid with a long idle wait. The protocol is still correct; the user has gone.
- Too short: the timer fires on a message that is still travelling, and copies pile up on a channel that is losing nothing.
- The floor is not a constant: at least one round trip, plus whatever the receiver takes to answer.
- And the worst case on a real path cannot even be estimated — queues make the delay differ from one packet to the next.
- So the value is chosen to make a loss likely, not certain: the protocol is built knowing the timer will sometimes be wrong.

How wrong can it be before the protocol stops working?

A timeout shorter than the RTT

The timer is shorter than the round trip. What can the sender do right?

Nothing. The acknowledgement cannot arrive before the timer fires, so every message is retransmitted — on a channel that is losing nothing.



nothing was lost, nothing was corrupted — and B has two copies

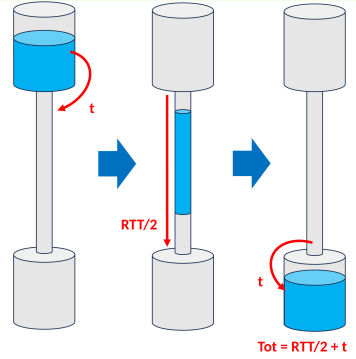
This is the 50 ms timer on the 80 ms path: everything goes twice, and the sequence number at the receiver is the only reason the program still works.

From a perfect channel to a real one

What the channel does	What the protocol must add
nothing goes wrong	nothing: make the packet and send it
bits can be corrupted	checksum, acknowledgements, retransmission
the acknowledgement too	a sequence number, so a copy is recognisable
packets can be lost	a timer at the sender, and the retransmission it triggers

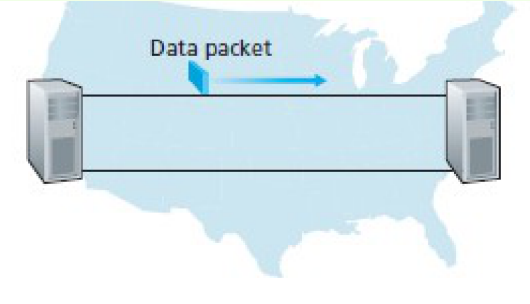
The last row is the alternating-bit protocol: checksum, one-bit sequence number, numbered acknowledgements, one timer. It is correct on any channel we have described — and we are about to measure how slow.

Filling a pipe with water



Pouring the tank into the tube takes t: that is transmission, L/R, and it depends on how *wide* the tube is. Then the water has to cross it — propagation, $RTT/2$, which depends only on how *long* it is. The last drop lands at $RTT/2 + t$.

Coast to coast: eight microseconds



- Two hosts on opposite coasts of the United States: the speed-of-light round trip is about 30 ms.
- A 1 Gbps channel, packets of 1000 bytes: $t = L/R = 8000/10^9 = 8\ \mu s$.
- The last bit arrives at $RTT/2 + L/R = 15.008$ ms, and the acknowledgement is back at $RTT + L/R = 30.008$ ms.

The sender is idle 99.97% of the time

Sender utilisation

The fraction of time the sender is actually pushing bits into the channel:

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} = 0.00027$$

- In 30 ms it sent 1000 bytes: an effective throughput of 267 kbps over a link that can do 1 Gbps.
- Nothing is broken and nothing was lost. The protocol is simply sending one message per round trip.
- Which means the limit does not move when you buy a faster link: it moves when you change the protocol.

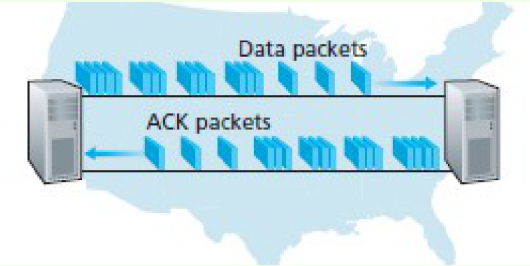
Example: A stop-and-wait chat

The formula

One message, one acknowledgement, then the next. On a path with a 100 ms round trip, a 1 KB message and a 100 Mbps link: $L/R = 80\ \mu s$, so $U = 0.0008$ and the chat moves about 80 kbps.

- On loopback and on the laboratory network the round trip is a fraction of a millisecond, so one never sees it.
- Ten messages a second is fine for a chat. It is not a way to move a file, and it is not what TCP does.

Pipelining

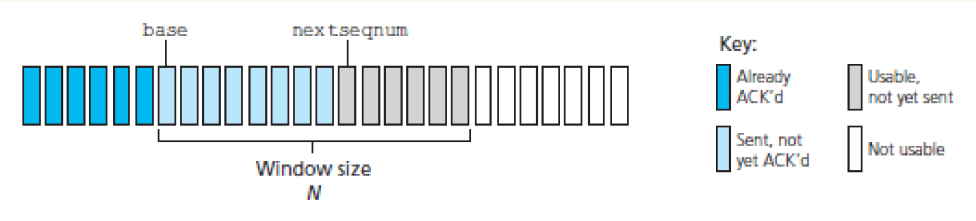


- Let the sender put several messages on the wire before waiting for anything to come back.
- With three in flight the sender is busy three times as long in the same 30 ms: $U \approx 0.00081$, about 800 kbps.
- And it keeps growing with the number in flight, until the channel is the limit again instead of the protocol.

What pipelining costs

- Sequence numbers: every packet in flight needs a different one, so a single alternating bit will not do any more.
- Buffers at the sender: everything sent and not yet acknowledged has to be kept: it may have to go out again.
- Buffers at the receiver: correct packets can now arrive with a hole in front of them, and something must be done with them.
- That last decision is the whole difference between the two protocols: Go-Back-N throws them away, Selective Repeat keeps them.

Go-Back-N: the sender's window



- `base` is the sequence number of the oldest unacknowledged packet; `nextseqnum` the smallest unused one.
- The sender may transmit without waiting, but never more than N unacknowledged packets — the window, which slides forward as acknowledgements arrive.

Four intervals, and a window that can be full

Sequence numbers	**State**
$[0,\ \text{base} - 1]$	transmitted and acknowledged
$[\text{base},\ \text{nextseqnum} - 1]$	sent, waiting for an acknowledgement
$[\text{nextseqnum},\ \text{base} + N - 1]$	may be sent right now
$[\text{base} + N,\ \infty)$	unusable until the window moves

When the window is full the sender refuses the data and hands it back up. A transport layer that says *not now* to the application is new — and it is where flow control will come from.

Cumulative acknowledgements, and a single timer

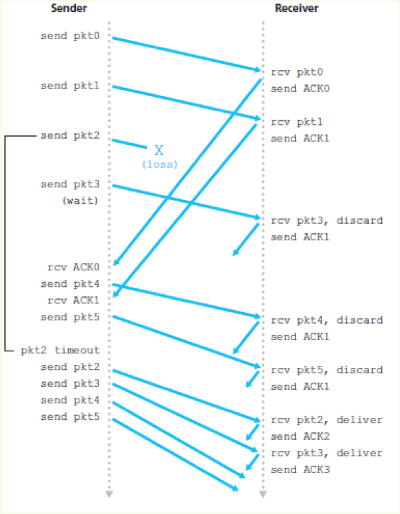
- An acknowledgement for packet *n* means: everything up to and including *n* arrived correctly.
- So a lost acknowledgement repairs itself — ACK 5 makes ACK 3 and ACK 4 unnecessary, and the window jumps forward.
- There is one timer, and it belongs to the oldest unacknowledged packet, not to each packet.
- On timeout the sender resends all the outstanding packets: that is what the protocol is named after.
- When an acknowledgement arrives the timer is restarted if anything is still outstanding, and stopped if nothing is.

The receiver keeps one number

- Its entire state is `expectedseqnum`: the sequence number of the next packet it is willing to deliver.
- Correct and in order: deliver it to the application, acknowledge it, move on by one.
- Anything else: discard it, and re-acknowledge the last in-order packet received.
- No buffer. A perfectly good packet that arrives early is thrown away exactly as if it had been damaged.
- The complexity is deliberately all on one side: the sender keeps a window, the receiver keeps an integer.

Go-Back-N in operation

- Window of four: packets 0 to 3 go out, then the sender has to wait for an acknowledgement to move the window.
- pkt2 is lost. Packets 3, 4 and 5 arrive correct and are discarded, because 2 has not been delivered.
- The receiver keeps answering `ACK1`: the last thing it actually handed to the application.
- The timer of pkt2 expires, and four packets go out again.

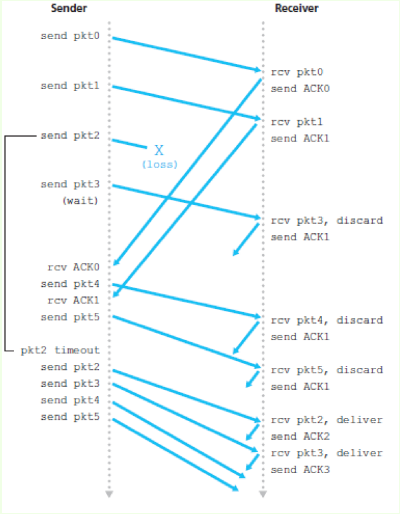


One loss, N retransmissions

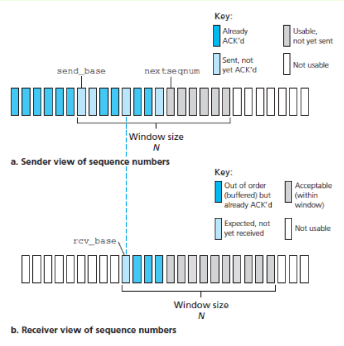
Three correct packets were thrown away. Who pays?

The sender and the channel, which carry them a second time.

- A chain reaction: the more the window holds, the more a single loss costs.
- Worst exactly where pipelining is worth most — a large window on a fast, long path.
- And a retransmission can be lost too, which starts it again.



Selective repeat: two windows that do not coincide



- Retransmit only the packets that are suspected of having been lost, so every packet is acknowledged individually.
- Out-of-order packets are buffered instead of discarded, and a window of N still bounds what is outstanding.

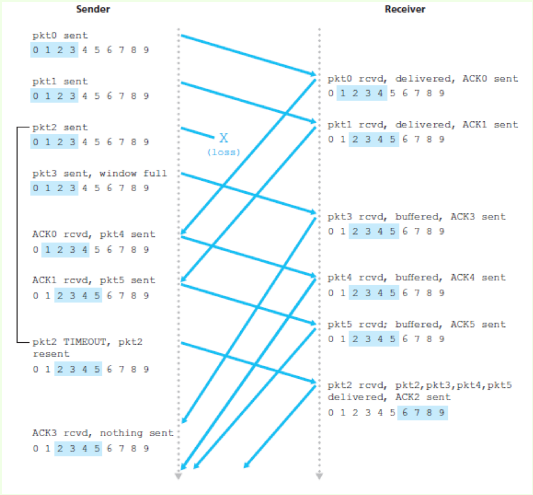
What each side does

Sender	**Receiver**
Data from above, if the number falls inside the window: packetise and send it.	A packet inside the window: acknowledge it, and buffer it if it is new.
One timer *per packet*. On timeout, resend that packet and no other.	If it is the base of the window: deliver it and every buffered packet consecutive with it.
ACK n: mark n received; if n was the base, slide to the next unacknowledged number.	A packet just *below* the window: acknowledge it again, and drop it.

The timers are what make it expensive: N of them, where Go-Back-N keeps one.

Selective repeat in operation

- Same window, same loss: pkt2 disappears while 3, 4 and 5 arrive correctly.
- Each of them is acknowledged and kept, even though nothing can be delivered yet.
- Only pkt2 is retransmitted, on its own timer. Nothing else goes out twice.
- When it arrives, four packets go up to the application in one move, in order.



Acknowledge what is already below your window

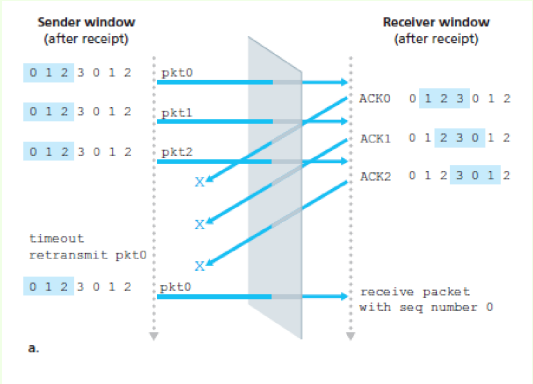
The receiver has that packet, delivered long ago. Why answer at all?

Because the sender does not know. If the acknowledgement for the oldest unacknowledged packet was lost, the sender will keep retransmitting it, and its window will never move forward.

- The two sides do not share a view of what has arrived, and in selective repeat that gap is permanent, not temporary.
- Silence is never an acceptable answer to a retransmission: it is indistinguishable from a second loss.
- So the receiver re-acknowledges and drops — for exactly this reason.

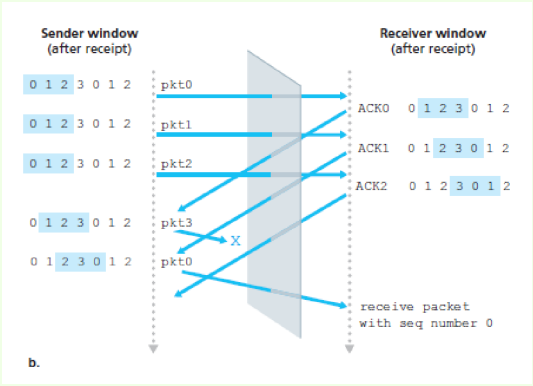
A window too large: the acknowledgements are lost

- Four sequence numbers, 0 to 3, and a window of three: one less than the space, which looks safe.
- Packets 0, 1, 2 arrive and are acknowledged, so the receiver's window moves on to $[3, 0, 1]$.
- Case a: all three acknowledgements are lost. The sender times out and retransmits pkt0.
- The receiver sees a packet numbered 0, and it is inside its window.



The same thing seen from behind the curtain

- Case b: the acknowledgements all arrive. The sender sends 3, 0, 1 — and pkt3 is lost while pkt0 gets through.
- The receiver sees a packet numbered 0, inside its window, and this time it is new data.
- It cannot see the sender: both cases produce the same sequence of arrivals, so the two are indistinguishable.
- Rule: the window must be at most half the sequence number space — and no larger.



One assumption we have been making all along

- Every channel model so far could corrupt and lose, but never reorder: what came out, came out in the order it went in.
- True of a single wire. Not true of a network, where two packets can follow different paths and arrive in the wrong order.
- Then an old copy of sequence number x can surface long after x was retired, and be accepted as if it were current.
- The defence is time: a number is not reused until the sender is sure that no old copy can still be alive out there.
- The Internet assumes a maximum packet lifetime of about three minutes — which is why TCP cannot simply start counting from zero.

The mechanisms, in one table

Mechanism	**What it is for**
Checksum	Detects bit errors inside a packet.
Timer	Fires when nothing came back, and the answer is always the same: send it again. Duplicates are the price.
Sequence number	A gap says something was lost; a repeated value says this is a copy.
Acknowledgement	Tells the sender what the receiver has — one packet at a time, or everything up to a number.
Negative acknowledgement	Says a packet did not arrive correctly. Unnecessary once acknowledgements carry a number.
Window, pipelining	Bounds what may be outstanding, and buys back the utilisation that stop-and-wait throws away.

Summary

- A reliable channel is rebuilt out of an unreliable one with checksums, sequence numbers, acknowledgements, timers and a window.
- Stop-and-wait is correct and costs one round trip per message: on a fast, long path it leaves the link almost idle.
- Pipelining buys that back, and forces wider sequence numbers, buffers, and a decision about out-of-order packets.
- Go-Back-N: cumulative acknowledgements, one timer, a receiver with no buffer — and one loss retransmits a whole window.
- Selective repeat: individual acknowledgements, a timer per packet, buffering — and a window no larger than half the sequence space.
- Next comes the protocol that is neither: cumulative acknowledgements as in Go-Back-N, buffering as in selective repeat.